2026 年 4 月 15 日メモ

コンピュータサイエンス入門

1　はじめに — コンピュータサイエンスは「速い機械」の学問ではない

コンピュータサイエンスというと, 多くの人は「プログラミング」や「AI」や「高性能なコンピュータ」を思い浮かべるかもしれない. もちろんそれらも重要である. しかし, この分野のもっと根本にある面白さは, **同じ問題でも, 考え方を変えるだけで必要な手間が大きく変わる**という点にある.

たとえば, たくさんの本を並べ替える, 欲しい情報を探す, 地図の上で最短の道を見つける, 長いデータを小さく保存する, といった作業を考えてみよう. どれも一見すると日常的で, 力任せでも一応はできる. しかし, 数が増えると, 素朴な方法はすぐ苦しくなる. そこで,

> 問題の中にどんな構造があるかを見つけ, その構造を利用して無駄を減らす

という発想が重要になる. これが, この文章全体を貫くコンピュータサイエンスの視点である.

以下では, まず「並び替え」と「探索」という最も分かりやすい例から出発し, そのあとで最短経路, 圧縮, 待ち時間, ハッシュ, 暗号, 機械学習へと話を広げていく. 読みながら注目してほしいのは, それぞれの技術の名前そのものではない. 大切なのは, **問題の見方を変えると世界が変わる**という感覚である.

2　まずは並び替え — 素朴な方法と賢い方法の差

コンピュータサイエンスの面白さを最も直感的に感じやすい題材の一つが, データの**並び替え**である.

たとえば, 本を題名順に並べる場面を考える. 何も知らずに「順番に整列させてください」と言われたら, 多くの人は次のような方法を思いつくだろう.

2.1　人が思いつきやすい素朴な方法

一つ目は, 左から順に見ていき, いま見ている本を適切な位置に差し込む方法である. これは挿入ソートと呼ばれる.

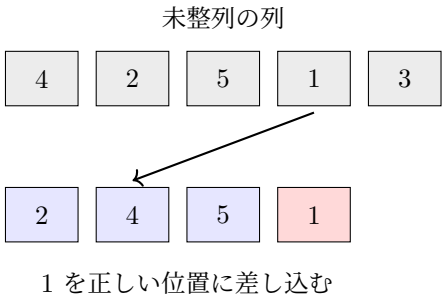


図 1　挿入ソートのイメージ

二つ目は, まだ並んでいない本の中から最も小さいものを探し, 先頭に置くことを繰り返す方法である. これは選択ソートと呼ばれる.

どちらも考え方としては自然で, 人が手でやる作業にも近い. そのため, 初めて聞いた人にも理解しやすい.

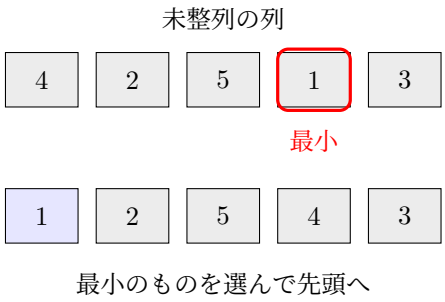


図 2　選択ソートのイメージ

2.2　冊数が増えると, 素朴さは弱点になる

問題は, 本の数が多くなったときである. たとえば 1000 冊の本を並べ替えるとする.

挿入ソートや選択ソートでは, 一冊ごとに多くの本と見比べる必要があるため, 最悪の場合の比較回数はだいたい

$$1000 \times 1000 = 1{,}000{,}000$$

回程度になる.

正確には $\frac{1000 \times 999}{2}$ 回程度であるが, ここでは「だいたい 100 万回」と考えてよい. 細かい式そのものよりも, **データの個数が増えると, 手間が急激に増える**ことが重要である.

100 冊ならまだ何とかなるかもしれないが, 1000 冊, 1 万冊と増えると, このやり方はすぐに限界を迎える.

2.3　賢い方法は, いきなり全部を片づけようとしない

ここでコンピュータサイエンスらしい発想が登場する. それは, **問題をそのまま丸ごと扱わず, 小さく分けながら解く**という考え方である.

2.3.1　マージソート — 分けてから整然と戻す

マージソートでは, まず本の山を半分に分ける. さらにそれぞれを半分に分ける. これを, 一冊ずつになるまで繰り返す.

一冊だけなら, それ以上並び替える必要はない. そこで今度は, 小さい順になるように二つずつ合体させていく. つまり, **小さく分けてから, 順序を保ちながら戻す**のである.

この方法のよいところは, 毎回「半分に分ける」という単純な操作で問題を小さくできる点にある. 大きな問題を, 扱いやすい小さな問題の集まりとして考えるのである.

2.3.2　クイックソート — 基準を決めて一気に分ける

クイックソートでは, まず適当な一冊を基準として選ぶ. そして, その本より前に来るべきものを左に, 後ろに来るべきものを右に集める.

すると, 基準に選んだ本は, その時点で「最終的に入るべき位置」に置かれることになる. あとは左側のグループと右側のグループに対して, 同じことを繰り返せばよい.

つまり, 一つの基準を使って全体を整理し, そのあと部分ごとに片付けていく方法である. マージソートが「分けてから丁寧に戻す」方法だとすれば, クイックソートは「基準を置いて一気に振り分ける」方法だと言える.

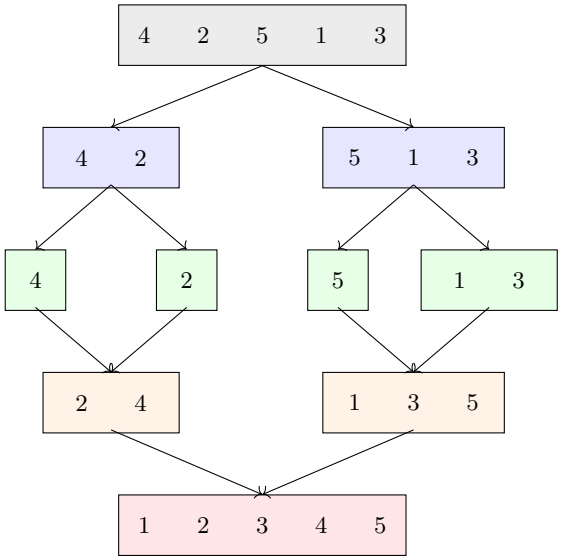


図 3　マージソートのイメージ

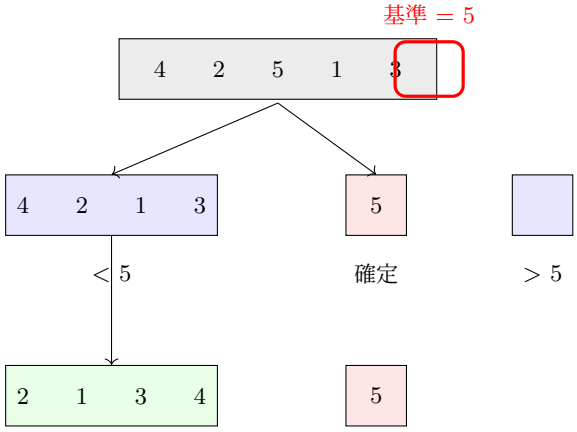


図 4　クイックソートのイメージ

2.4　どれくらい速くなるのか

マージソートやクイックソートでは, 必要な比較回数はおおよそ

$$n \log_2 n$$

に比例する.

1000 冊なら

$$1000 \times \log_2 1000 \approx 1000 \times 10 = 10{,}000$$

程度で済む.

つまり,

$$1{,}000{,}000 \quad \rightarrow \quad 10{,}000$$

であり, およそ **100 分の 1** になる.

ここで重要なのは, コンピュータが速いから解決したのではないという点である. **考え方そのものを変えたことで, 必要な手間が激減した**のである.

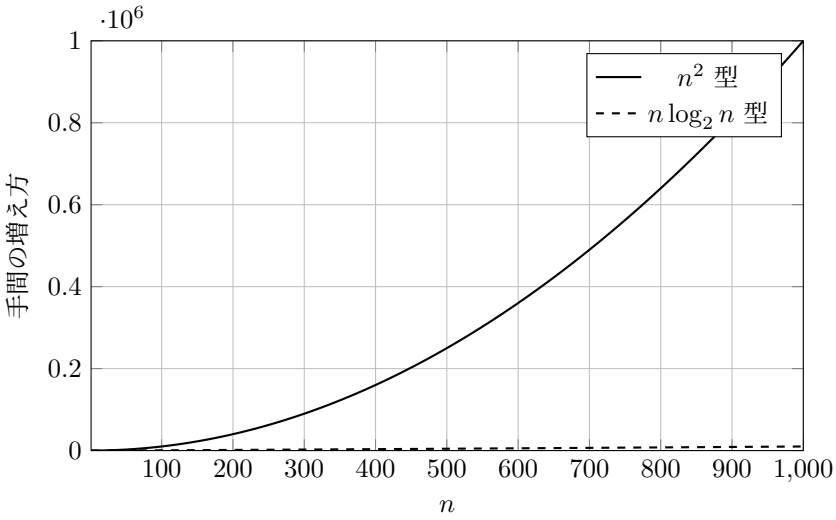


図 5　手間の増え方の違い

このグラフを見ると, 少し工夫しただけで, データが大きくなったときの差が決定的になることが分かる.

2.5　この例から見える本質

ソートは単なる並び替えの話に見えるかもしれない. しかし, その背後にはコンピュータサイエンスの本質が表れている.

同じ問題でも, 素朴に解くか, うまく分割して解くかで, かかる時間は大きく変わる. しかもその差は, データが大きくなるほど決定的になる. ここに, コンピュータサイエンスのおもしろさがある. それは, ただ計算する学問ではなく, **どう考えれば問題をもっと賢く解けるかを探る学問**だということである.

3　次は探索 — 全部を見るか, 半分ずつ捨てるか

並び替えの次に考えやすいのが, 欲しいものを**探す**という問題である.

たとえば, 1 から 1000 までの番号が順番に並んだカードの中から, ある番号を探したいとする. 目当ての番号がどこにあるかは分からない.

3.1　普通に探すとどうなるか

何も工夫しなければ, 先頭から 1 枚ずつ見ていくことになるだろう. これはとても自然な方法であり, 線形探索と呼ばれる.

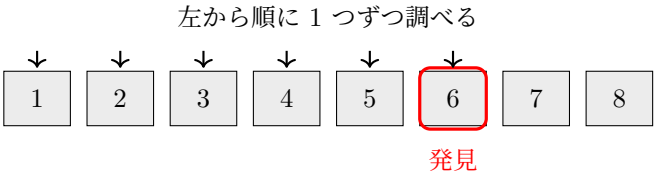


図 6　線形探索のイメージ

この方法は分かりやすいが, 数が多くなると大変である. 1000 個の中から探すとすると, 最悪では 1000 回近く調べることになる.

たまたま最初のほうにあればすぐ見つかるが, 最後のほうにあればほとんど全部を見る必要がある. つまり, 個数が増えると, 手間もほぼそのまま比例して増えていく.

3.2 並んでいるなら, その性質を使える

ここで大事なのは, カードが**順番に並んでいる**という事実である. この情報を使わないのはもったいない.

たとえば辞書で単語を探すとき, 1 ページ目から順に読む人はほとんどいない. まず真ん中あたりを開き, そこが目当てより前か後ろかを見て, 探す範囲を半分にするはずである. この考え方が二分探索である.

3.3 二分探索 — 一回ごとに半分を捨てる

二分探索では, まず真ん中を見る. 目当ての番号より小さければ, 左半分はもう見る必要がない. 反対に大きければ, 右半分も不要になる.

つまり, 1 回調べるたびに, 候補を半分にできる.

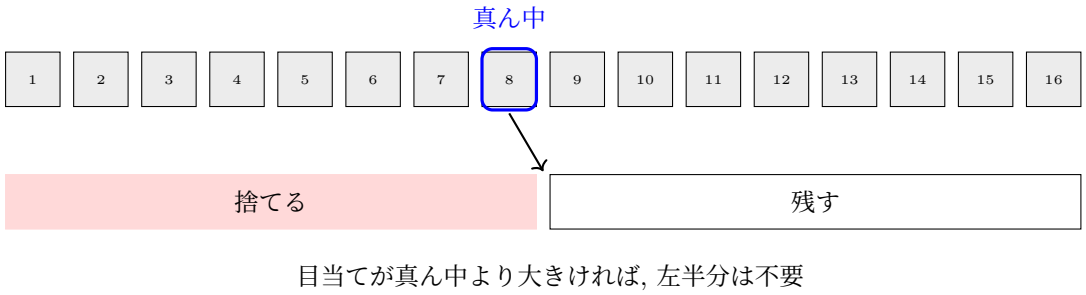


図 7　二分探索では, 真ん中を見て半分を捨てる

そして残った半分に対して, また真ん中を見る. これを繰り返すと, 候補はどんどん小さくなる.

たとえば 1000 個あるとしても,

$$1000 \to 500 \to 250 \to 125 \to 63 \to 32 \to 16 \to 8 \to 4 \to 2 \to 1$$

というように減っていく.

つまり, 1000 個あっても, 10 回くらい調べれば見つけられるのである.

3.4 どれくらい差がつくのか

普通に 1 個ずつ探す線形探索では, 最悪で

$$1000$$

回程度の確認が必要になる.

一方, 二分探索では

$$\log_2 1000 \approx 10$$

回程度で済む.

つまり,

$$1000 \quad \to \quad 10$$

であり, 必要な回数はおよそ **100 分の 1** になる.

ここでも重要なのは, 特別に速い機械を使ったわけではないという点である. **順番に並んでいるという性質を利用し, 探し方そのものを変えた**ことで, 圧倒的に速くなったのである.

3.5　ソートと探索に共通すること

ここまでで, すでにコンピュータサイエンスの重要な感覚が二つ見えている.

ソートでは, 全体を小さく分けることで速くした. 探索では, 必要のない半分を思い切って捨てることで速くした.

どちらも共通しているのは, **問題の構造を見つけ, それを利用して無駄を減らしている**ことである. このあとに出てくる題材も, 実はみなこの延長線上にある.

3.6　補足: 計算は読み飛ばしてよい

厳密には, 線形探索は $O(n)$, 二分探索は $O(\log n)$ と書かれる. ただし, ここで大切なのは記号そのものではない.

- 1 個ずつ見る方法では, データが 10 倍になれば, 手間もほぼ 10 倍になる
- 半分ずつ減らす方法では, データが大きくなっても, 手間はそれほど増えない

という感覚をつかむことが重要である.

4　最短経路 — 全部試すのではなく, 確定できるところから進む

ここまで見てきた「分ける」「捨てる」という発想は, 道を探す問題にも現れる.

たとえば, 地図アプリが, 今いる場所から目的地までの最短経路を示してくれる場面を考える. 人が感覚で道を選ぶなら,「なんとなく近そうな方向に進む」「大きい道を通ればよさそうだ」と判断するかもしれない. しかし, それだけでは本当に最短とは限らない.

4.1　素朴に考えると, 候補はすぐ増えすぎる

最も素朴な考え方は, 行けそうな道を全部試して比べることである. しかし, 分かれ道が何度もあると, 候補となる経路はすぐに大量に増えてしまう.

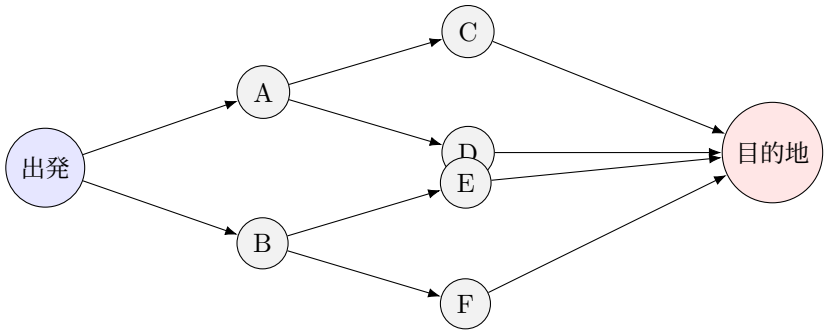


図 8　分かれ道が増えると, 試すべき経路はすぐに多くなる

道が少ししかなければこれでもよいが, 現実の道路網はもっと複雑である. 都市全体や国全体の地図でこれをやろうとすると, 全部試すのは現実的ではない.

4.2　ダイクストラ法 — いちばん近いものから確定する

ここで重要なのは,「今のところ最も近い場所から順に確定していく」という考え方である. これを用いる代表的な方法が, ダイクストラ法である.

出発点から見て, まず一番近い場所はすぐに分かる. その場所までの最短距離は, 後から別の遠回りを考えても, それより短くなることはない. そこで, 一番近い場所を一つずつ確定しながら, 周囲の候補の距離を更新していく.

4.3 ダイクストラ法のイメージ

たとえば, 次のような地図を考える.

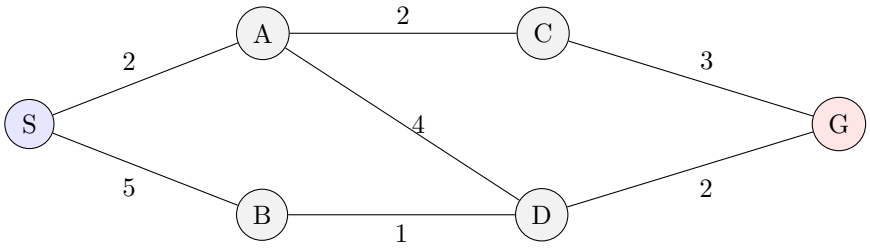


図 9　重み付きの道の例

ここで, S から G までの最短経路を知りたいとする.

まず S から直接行けるのは A と B なので, A までの距離は 2, B までの距離は 5 だと分かる. この時点では, A のほうが近いので, まず A を確定する.

A を経由すると,

$$S \to A \to C$$

で C までの距離は 4 になる. 同様に,

$$S \to A \to D$$

で D までの距離は 6 になる.

次に, まだ確定していない候補の中で最も近いのは C である. そこで C を確定し, G までの距離候補を

$$4 + 3 = 7$$

と更新する.

一方で, B を通る道も考えられる. B までは 5, そこから D まで 1 なので,

$$S \to B \to D$$

なら D まで 6 である. これは先ほどの候補と同じである.

最後に D から G へ行くと,

$$6 + 2 = 8$$

となるので, G までの最短距離は 7 だと分かる. したがって最短経路は

$$S \to A \to C \to G$$

である.

4.4 この方法のうまさ

この方法のうまいところは, 毎回「今いちばん近い場所」だけに注目する点にある. 全部の道を最後まで試してから比べるのではなく, **確実に正しいものから順に固めていく**のである.

ここでも, コンピュータサイエンスらしい発想は同じである.

全部やってから考えるのではなく, 今の情報だけで確定できる部分を見つけて前に進む

ソートでは「分ける」ことが重要だった. 探索では「半分を捨てる」ことが重要だった. 最短経路では「確定できるものから順に確定する」ことが重要になる.

4.5 補足: 計算は読み飛ばしてよい

厳密な計算量の議論では, ダイクストラ法はデータ構造によって効率が変わる. ただし, ここで大切なのは式そのものではない.

- 思いつくまま全部の道を試すと, 候補はすぐ爆発的に増える
- しかし, うまい規則を使うと, 必要な計算を大きく減らせる

という感覚をつかむことが重要である.

5 データ圧縮 — 情報を減らさず, 表し方を賢くする

ここまでは主に「速く解く」話だったが, コンピュータサイエンスの面白さは「小さく表す」ことにも現れる. その典型例がデータ圧縮である.

たとえば, 長い文章や画像や音声をそのまま保存すると, 多くの容量が必要になる. しかし実際には, zip ファイルや画像圧縮, 音声圧縮によって, 同じ内容をもっと小さく保存できることが多い.

5.1 そのまま保存するのは確実だが, 無駄も多い

最も素朴な方法は, データをそのまま全部書いて保存することである. たとえば, 次のような文字列を考える.

$$AAAAAAABBBBBBBBBCCCCCC$$

これをそのまま保存するなら, 文字を一つずつ全部記録する必要がある.

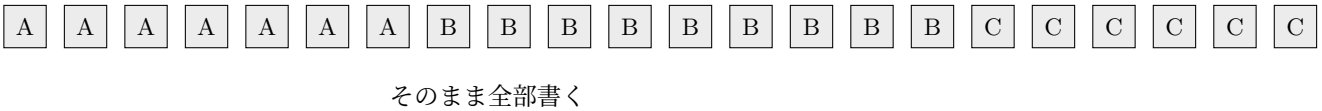


図 10 圧縮しない保存

これは確実だが, 同じ文字が何回も続くような場合には, 少し無駄に見える.

5.2 規則を見つけると, もっと短く書ける

ここで,

$$AAAAAAA$$

という並びを見たとき, これは「A が 7 回続く」とまとめて書けそうだと気づく. 同じように,

$$BBBBBBBBB$$

は「B が 9 回続く」,

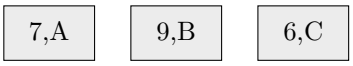
$$CCCCCC$$

は「C が 6 回続く」と表せる.

すると, 元の文字列は

$$(7, A)(9, B)(6, C)$$

のように書ける.



繰り返しをまとめて書く

図 11　連続する部分をまとめる圧縮の例

この考え方は, 連長圧縮と呼ばれる. もちろん, 実際の圧縮技術はもっと複雑だが, 基本の発想は同じである. つまり, データの中にある**規則性や繰り返し**を見つけて, そこを短く表すのである.

5.3　なぜ小さくできるのか

圧縮の本質は, 情報そのものを消しているわけではなく, **同じ内容をもっと要領よく書き直している**ことにある.

たとえば人間でも, 長い説明をそのまま丸暗記するより,「これは同じパターンの繰り返しだ」とまとめて覚えたほうが楽なことがある. 圧縮は, それをデータに対して行っているとも言える.

5.4　どんなときに効くのか

規則が多いデータほど, 圧縮はよく効く. たとえば,

- 同じ文字や記号が何度も出るデータ
- 同じ色の部分が広く続く画像
- 似た音が続く音声

では, 元のまま保存するよりずっと小さくできることがある.

一方で, もともとばらばらで規則が少ないデータは, あまり小さくならない. つまり, 圧縮とは何でも魔法のように小さくする技術ではなく, **データの中の規則性を見つけて利用する技術**なのである.

5.5　ここまでとの共通点

ソートでは, 並び替え方を工夫することで手間を減らした. 探索では, 順序の情報を使って不要な候補を捨てた. 圧縮では, 規則性を見つけることで保存量を減らした.

どの場合も共通しているのは,

> ただ力任せに処理するのではなく, 問題の中の構造を利用して, 必要な手間や量を減らしている

という点である.

5.6　補足: 計算は読み飛ばしてよい

実際の圧縮技術には, ハフマン符号, LZ 系列, JPEG, MP3 などさまざまな方式がある. ただし, 最初に大切なのは細かい名前ではない.

- データには規則があることがある
- 規則があるなら, そのままより短く表せることがある

- その工夫が, ファイルサイズや通信量を大きく減らす

という発想そのものが重要である.

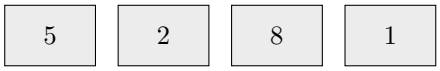
6　待ち時間 — 何をするかだけでなく, どの順番でやるか

コンピュータサイエンスのおもしろさは,「同じ仕事でも順番を変えるだけで効率が大きく変わる」点にもある. そのことを, 待ち時間の問題で考えてみよう.

たとえば, 1 台のプリンタに複数の印刷ジョブが並んでいるとする. それぞれのジョブには, 印刷にかかる時間がある.

6.1　到着順に処理すると何が起こるか

何も考えなければ, 到着した順に処理するだろう. これは FIFO（先に来たものから処理）である.



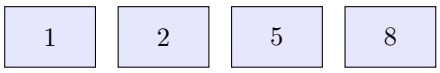
到着順に処理

図 12　FIFO の例（数字は処理時間）

しかし, 先頭に時間の長いジョブがあると, 後ろの短いジョブはずっと待たされてしまう.

6.2　短い仕事を先にすると, 全体は軽くなる

ここで,「短い仕事から先に処理する」とどうなるか考える. これを最短ジョブ優先と呼ぶ.



短いものから処理

図 13　最短ジョブ優先

すると, 短いジョブはすぐ終わるため, 全体としての待ち時間が大きく減る.

6.3　ここで面白いのは何か

この方法では, 問題そのものを変えたわけではない. 変えたのは, ただ**順番**だけである. それなのに, 全体の効率は大きく改善される.

つまり重要なのは,

> 何をするかだけでなく, どの順番でやるかも設計する

という点である.

これもまた, コンピュータサイエンスが単なる計算作業ではなく, 処理の設計そのものを考える学問であることを示している.

7　ハッシュ — そもそも「探さない」という発想

探索の話をさらに進めると, もっと大胆な発想にたどり着く. それは, **探すのではなく, 最初から場所を当てる**という考え方である.

7.1　普通に探せば, やはり時間がかかる

名前と電話番号の一覧から, ある人の番号を探すとする. 何も工夫しなければ, 上から順に見ていくことになる. これは線形探索であり, データが増えると時間がかかる.

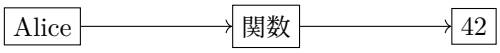
7.2　名前から直接場所を決める

ここで, 名前から直接場所を決められたらどうなるか考える.
たとえば,

- 名前を数字に変換する
- その数字を使って配列の位置を決める

とすれば, 探すのではなく, 最初からそこを見ればよい.

7.3　ハッシュのイメージ



名前から直接位置を決める

図 14　ハッシュの基本イメージ

このとき使う「変換ルール」をハッシュ関数と呼ぶ.

7.4　何がすごいのか

この方法では, データの数が増えても, 探す回数はほとんど増えない.
つまり,

> たくさんの中から探すのではなく, 最初から場所を当てる

という発想になっている.
これまでの方法は,

- 順番に見る
- 半分に減らす

だった.
しかしハッシュでは, 極端に言えば

- 探さない

のである.

この発想の転換こそが, コンピュータサイエンスの面白さである.

7.5 補足: 完璧ではないが, それでも強い

同じ場所に複数のデータが入ってしまうこともある. これを衝突と呼ぶ. その場合は少し工夫が必要になる.

しかしそれでも, 平均的には非常に高速に動作する. つまりハッシュは, **探す問題を「計算して当てる問題」に変えてしまう**方法だと言える.

8 暗号 — 情報の形を変えることで安全を作る

ここまでの話は主に「速さ」や「効率」に関するものだった. しかしコンピュータサイエンスのおもしろさは, 情報を**安全に扱う**ための工夫にも現れる. その代表例が暗号である.

たとえば, インターネットでメッセージを送るとき, そのまま送れば途中で盗み見られてしまう可能性がある.

8.1 そのまま送ると読まれてしまう

メッセージをそのまま送ると, 通信経路のどこかで見られた場合, 内容はすぐに読まれてしまう.

そのまま送ると読めてしまう

図 15 平文通信

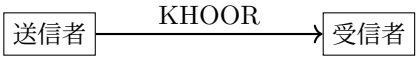
8.2 読めない形に変えてから送る

そこで, メッセージを別の形に変換してから送ることを考える.

たとえば, アルファベットをずらすことで

$$HELLO \to KHOOR$$

のように変換する.



読めない形に変換して送る

図 16 暗号化の基本

受信者は同じルールを使って元に戻せる.

8.3 現代の暗号の発想

単純な変換では, すぐに破られてしまう. そこで, より強い仕組みが必要になる.

現代の暗号では,

- 鍵がないと復元できない
- しかし公開された情報だけでも暗号化はできる

という仕組みが使われる.

8.4 公開鍵のイメージ

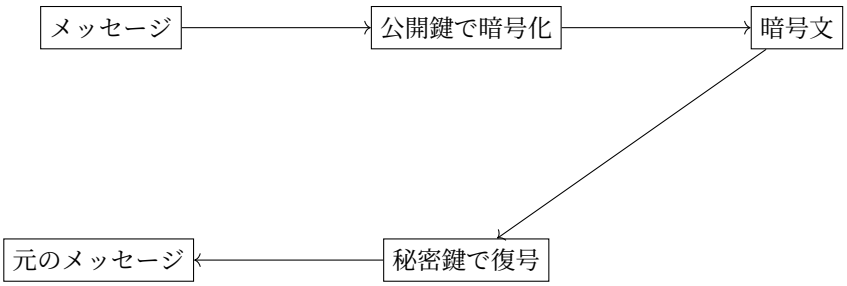


図 17　公開鍵暗号のイメージ

ここでのポイントは,

　　暗号化と復号で使う情報を分ける

ことである.

暗号の面白さは,

- 誰でも公開鍵を使って送れる
- しかし正しい秘密鍵を持つ人だけが読める

という, 一見矛盾したことを実現している点にある. これは単なる工夫ではなく, 数学的な性質を利用して作られている.

8.5 この例が示すこと

暗号は, **情報を守るために, 情報の形そのものを変える**技術である. ここにも,

　　問題の構造を利用して, できなかったことを可能にする

というコンピュータサイエンスの本質が現れている.

9 機械学習 — ルールを書くのではなく, ルールを学ばせる

最後に, 現代のコンピュータサイエンスを語るうえで欠かせない題材として機械学習を見る.

ここまでに登場した多くの方法では, 人間が「どう解くか」を考え, 手順を設計していた. しかし, 画像認識のように, 人間がうまくルールを書けない問題もある.

9.1 普通のプログラムの考え方

従来のプログラムでは,

- 入力
- ルール

- 出力

という形で処理を行う.

たとえば, 数式を計算するプログラムでは, 計算方法というルールを人間が書いている.

9.2 しかし, ルールを人が書きにくい問題もある

画像認識のような問題では,「猫とは何か」というルールを人間が完全に書くのは難しい.

耳の形, 目の形, 色, 姿勢など, 条件が複雑すぎるためである.

9.3 発想の転換 — 例から学ばせる

そこで, 発想を逆にする.

- 入力と出力の例をたくさん与える
- コンピュータにルールを学ばせる

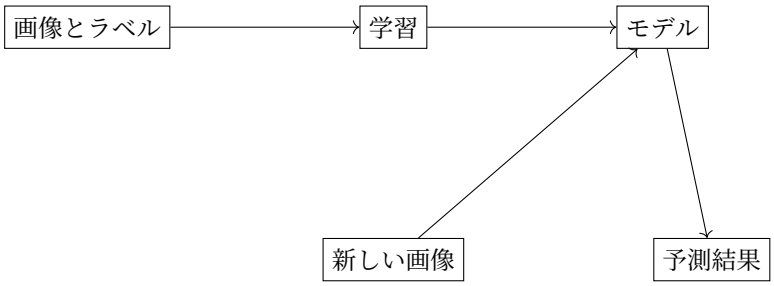


図 18　機械学習の基本構造

この方法では, コンピュータはデータの中にあるパターンを見つけている. たとえば,

- 猫の画像には似た特徴がある
- 犬の画像には別の特徴がある

といった違いを, 数値として学習している.

9.4 なぜこれが面白いのか

ここで重要なのは,

> 人間がルールを与えるのではなく, コンピュータがルールを作る

という点である.

これは従来のプログラムとは大きく異なる. 機械学習は, 問題を「ルールを書く問題」から「データから学ぶ問題」に変える技術だと言える.

9.5 補足

実際には, ニューラルネットワークなどの複雑な仕組みが使われている. しかし最初に重要なのは,

- データからパターンを学べる

- ルールを人間が書かなくてもよい

という発想そのものである.

10　おわりに — さまざまな話題を貫く一つの考え方

ここまで, 並び替え, 探索, 最短経路, 圧縮, 待ち時間, ハッシュ, 暗号, 機械学習を見てきた. それぞれは別々の話題に見えるかもしれない.

しかし, それらを貫いている考え方は共通している.

- ソートでは, 問題を小さく分けた
- 探索では, 不要な半分を捨てた
- 最短経路では, 確定できるものから順に固めた
- 圧縮では, 規則性を見つけて表現を短くした
- 待ち時間では, 処理の順番を工夫した
- ハッシュでは, 探すこと自体を別の問題に置き換えた
- 暗号では, 情報の形を変えて安全を作った
- 機械学習では, ルールを書く代わりに, 例からルールを学ばせた

つまりコンピュータサイエンスとは, ただ計算機を動かすための技術ではない. **問題の見方を変え, 構造を見抜き, より少ない手間で, よりうまく解く方法を設計する学問**なのである.

この視点を持つと, コンピュータサイエンスは「難しい数式の世界」というよりも, むしろ「考え方の工夫が世界を変える学問」として見えてくる. そこにこそ, この分野の大きな魅力がある.